

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2**



Stack and Queue

Oleh:

Rahma Syarita Jaya NIM. 2510817320004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack and Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Rahma Syarita Jaya
NIM : 2510817320004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1: Konversi Adjacency List ke Adjacency Matrix	6
A. Source Code.....	7
B. Output Program	8
C. Pembahasan	8
SOAL 2: Konversi Adjacency Matrix ke Adjacency List.....	10
A. Source Code.....	12
B. Output Program	13
C. Pembahasan	13
Soal 3: BFS: Jarak Terpendek Keluar Labirin.....	14
A. Source Code.....	15
B. Output Program	17
C. Pembahasan	17
Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin.....	19
A. Source Code.....	21
B. Output Program	22
C. Pembahasan	22
Soal 5: Tree Traversal.....	23
A. Source Code.....	25
B. Output Program	27
C. Pembahasan	27
TAUTAN GIT	29

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Modul 6	8
Gambar 2. Screenshot Hasil Jawaban Soal 2 Modul 6	13
Gambar 3. Screenshot Hasil Jawaban Soal 3 Modul 6	17
Gambar 4. Screenshot Hasil Jawaban Soal 4 Modul 6	22
Gambar 5. Screenshot Hasil Jawaban Soal 5 Modul 6	27

DAFTAR TABEL

Tabel 1. Source code prob1.cpp Modul 6.....	7
Tabel 2 Source code prob2.cpp Modul 6.....	12
Tabel 3. Source code prob3.cpp Modul 6.....	15
Tabel 4. Source code prob4.cpp Modul 6.....	21
Tabel 5. Source code prob5.cpp Modul 6.....	25

SOAL 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi **adjacency matrix**. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
M
U1 V1 W1 U2 V2 W2
...
UM VM WM
```

Keterangan.

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

```
Adjacency Matrix:
V1 V2 ... VN
V1 a11 a12 ... a1N
V2 a21 a22 ... a2N
...
VN aN1 aN2 ... aNN
```

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai

a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A.

Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

A. Source Code

Tabel 1. Source code prob1.cpp Modul 6

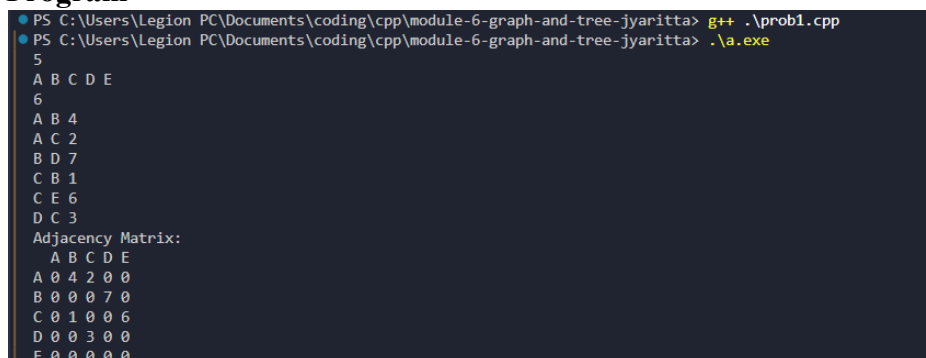
1	#include <iostream>
2	
3	using namespace std;
4	
5	int getIndex(char v, char vertices[], int n) {
6	for (int i = 0; i < n; i++) {
7	if (vertices[i] == v) return i;
8	}
9	return -1;
10	}
11	
12	int main() {
13	int n;
14	cin >> n;
15	
16	char vertices[15];
17	for (int i = 0; i < n; i++) {

```

18         cin >> vertices[i];
19     }
20
21     int m;
22     cin >> m;
23
24     int matrix[15][15] = {0};
25
26     for (int i = 0; i < m; i++) {
27         char u, v;
28         int w;
29         cin >> u >> v >> w;
30
31         matrix[getIndex(u, vertices, n)][getIndex(v,
vertices, n)] = w;
32     }
33
34     cout << "Adjacency Matrix:\n ";
35
36     for (int i = 0; i < n; i++) {
37         cout << vertices[i] << " ";
38     }
39     cout << "\n";
40     for (int i = 0; i < n; i++) {
41         cout << vertices[i];
42         for (int j = 0; j < n; j++) {
43             cout << " " << matrix[i][j];
44         }
45         cout << "\n";
46     }
47
48     return 0;
49 }

```

B. Output Program



```

PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> g++ .\probl1.cpp
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> .\a.exe
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0

```

Gambar 1. Screenshot Hasil Jawaban Soal 1 Modul 6

C. Pembahasan

Pada baris [1 sampai 2] `stack.cpp`, mengimpor file header "`stack.h`" untuk mengenali `struct Stack` dan mengimpor library `<stdexcept>` untuk memunculkan pesan error dengan `throw`.

Algoritma ini mengubah adjacency list yang menggunakan susunan daftar list menjadi adjacency matrix atau tabel dua dimensi berukuran $N \times N$. Inputnya ada N yaitu jumlah vertex, nama vertex, jumlah edge dan weight dari edge tersebut dan outputnya berupa graph representasi matriks.

Fungsi `getIndex` mencari indeks dari char vertex pada array vertices dengan menggunakan loop yang mengecek satu per satu elemen array. Jika nilai elemen sama dengan karakter yang dicari, kembalikan nilai indeksnya. Kalau tidak ketemu, return -1 untuk mengirimkan error. Time complexitynya adalah $O(N)$ karena melakukan iterasi sebanyak N , sedangkan space complexity adalah $O(1)$ karena tidak menggunakan memori atau fungsi tambahan.

Di fungsi `main`, menerima input nilai N dan nama-nama vertex ke array `vertex`. Lalu menerima input nilai M untuk jumlah edge untuk banyaknya sisi. Setelah itu inisialisasi array 2D matrix berukuran 15×15 yang berisikan 0. Lalu melakukan loop dari 0 sampai M untuk menambah edge ke graph.

Di dalam loop tersebut, program memanggil fungsi `getIndex` untuk mencari posisi baris dan kolom yang tepat, lalu memasukkan nilai weight edge ke dalam matrix dengan `matrix[getIndex(u, vertices, n)][getIndex(v, vertices, n)] = w;`. Terakhir, mencetak Adjacency Matrix, baris nama vertex dan print output graph melalui loop. Untuk time complexity-nya $O(M \times N + N^2)$ karena terdapat loop M untuk input edge dan di dalamnya memanggil fungsi `getIndex` dengan time complexity $O(N)$, ditambah nested loop untuk mencetak graph $O(N^2)$.

SOAL 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

V1 V2 ... VN A11 A12 ... A1N A21 A22 ... A2N

...

AN1 AN2 ... ANN

Keterangan:

-N adalah jumlah vertex.

- V_i adalah label vertex ke- i .

- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .

-Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .

-Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

V1 -> (X1,w1), (X2,w2), ...

V2 -> (X1,w1), (X2,w2), ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

$$-2 \leq N \leq 10$$

$$- 0 \leq A_{ij} \leq 1000$$

-Label vertex berupa karakter alfabet kapital A sampai Z.Semua label vertex dijamin unik.

- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5	Adjacency List:
A B C D E	A -> (B,4), (C,2)
0 4 2 0 0	B -> (D,7)
0 0 0 7 0	C -> (B,1), (E,6)
0 1 0 0 6	D -> (C,3)
0 0 3 0 0	E -> -
0 0 0 0 0	

Penjelasan Contoh

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B,4), (C,2).

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

A. Source Code

Tabel 2 Source code prob2.cpp Modul 6

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     int n;
7     cin >> n;
8
9     char vertices[15];
10    for (int i = 0; i < n; i++) {
11        cin >> vertices[i];
12    }
13
14    int matrix[15][15];
15    for (int i = 0; i < n; i++) {
16        for (int j = 0; j < n; j++) {
17            cin >> matrix[i][j];
18        }
19    }
20
21    cout << "Adjacency List:\n";
22
23    for (int i = 0; i < n; i++) {
24        cout << vertices[i] << " -> ";
25
26        bool isFirst = true;
27        for (int j = 0; j < n; j++) {
28            if (matrix[i][j] > 0) {
29                if (!isFirst) {
30                    cout << ", ";
31                }
32                cout << "(" << vertices[j] << ", " <<
matrix[i][j] << ")";
33                isFirst = false;
34            }
35        }
36        if (isFirst) {
37            cout << "-";
38        }
39        cout << "\n";
40    }
41
42    return 0;
43 }
```

B. Output Program

```
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> g++ .\prob2.cpp
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> .\a.exe
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
```

Gambar 2. Screenshot Hasil Jawaban Soal 2 Modul 6

C. Pembahasan

Algoritma ini mengubah adjacency matrix menjadi adjacency list. Inputnya ada nilai N untuk jumlah vertex, daftar nama vertex, dan graph yang dalam bentuk adjacency matrix atau tabel berukuran $N \times N$.

Pada fungsi `main()` pertama-tama melakukan input nilai N dan membaca nama-nama vertex atau simpul ke dalam array `vertices`. Setelah itu, lakukan inisialisasi array matrix berukuran 15×15 dengan nested loop i dan j dari 0 sampai N untuk menaruh input nilai bobot tadi ke dalam array matrix.

Kemudian untuk mencetak output, program melakukan nested loop lagi. Loop luar i digunakan untuk iterasi vertex dengan `vertices[i]` diikuti dengan tanda `"->"`. Setelah itu loop berikutnya di dalam loop luar, terdapat variabel boolean `isFirst` bernilai `true` untuk mengatur penempatan tanda koma pada nilai pertama yang muncul. Permisalan, pertama program akan mengecek apakah `isFirst` aktif, karena nilai yang pertama tercetak maka koma tidak ikut tercetak. Setelah nilai pertama keluar maka nilai `isFirst` akan jadi `false`, setelah ini program akan mencetak koma. Jika loop j selesai dan `isFirst` masih bernilai `true` Dimana vertex awal tidak memiliki jalur sama sekali. Jadi, program hanya akan mencetak tanda `"-"`.

Time complexity untuk algoritma ini sendiri $O(N^2)$ karena bergantung pada nested loop yang matriks $N \times N$ dengan space complexity $O(1)$ karena ukuran matrix sudah ditetapkan di awal sehingga tidak berubah-ubah sesuai nilai input N .

Soal 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

```
X
```

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$

- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
<pre> 8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9 </pre>	16

Penjelasan Contoh

Posisi awal berada pada $(0, 0)$ dan posisi tujuan berada pada $(7, 9)$. Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel $(2, 1)$.

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari $(0, 0)$ ke $(7, 9)$ adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 3. Source code prob3.cpp Modul 6

1	#include <iostream>
2	
3	using namespace std;
4	
5	struct State {
6	int r, c, dist;
7	};
8	
9	State q[10001];
10	int head = 0;
11	int tail = 0;
12	
13	void push(int r, int c, int d) {
14	q[tail].r = r;
15	q[tail].c = c;
16	q[tail].dist = d;
17	tail++;

```

18 }
19
20 State pop() {
21     State s = q[head];
22     head++;
23     return s;
24 }
25
26 bool isEmpty() {
27     return head == tail;
28 }
29
30 int grid[101][101];
31 bool visited[101][101];
32
33 int dr[] = {-1, 1, 0, 0};
34 int dc[] = {0, 0, -1, 1};
35
36 int main() {
37     int R, C;
38     cin >> R >> C;
39
40     for (int i = 0; i < R; i++) {
41         for (int j = 0; j < C; j++) {
42             cin >> grid[i][j];
43             visited[i][j] = false;
44         }
45     }
46
47     int SR, SC, FR, FC;
48     cin >> SR >> SC;
49     cin >> FR >> FC;
50
51     push(SR, SC, 0);
52     visited[SR][SC] = true;
53
54     int ans = -1;
55
56     while (!isEmpty()) {
57         State curr = pop();
58
59         if (curr.r == FR && curr.c == FC) {
60             ans = curr.dist;
61             break;
62         }
63
64         for (int i = 0; i < 4; i++) {
65             int nr = curr.r + dr[i];
66             int nc = curr.c + dc[i];
67
68             if (nr >= 0 && nr < R && nc >= 0 && nc < C) {
69                 if (grid[nr][nc] == 0 && !visited[nr][nc]) {

```



```

68         visited[nr][nc] = true;
69         push(nr, nc, curr.dist + 1);
70     }
71 }
72 }
73 }
74 cout << ans << "\n";
75 return 0;
76 }
77
78

```

B. Output Program

```

PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> .\a.exe
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16

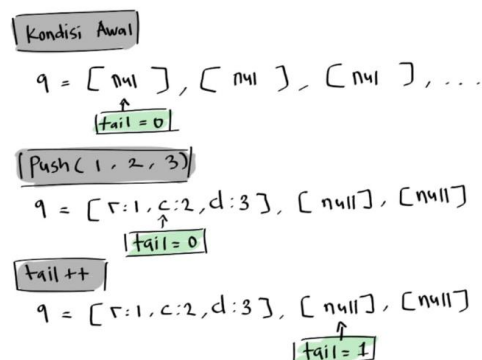
```

Gambar 3. Screenshot Hasil Jawaban Soal 3 Modul 6

C. Pembahasan

Algoritma berikut pencarian rute terpendek dalam sebuah labirin $R \times C$ menggunakan metode Breadth-First Search (BFS). Terdapat struct State yang berfungsi untuk menyimpan posisi baris r , kolom c , dan jumlah jarak $dist$. Selanjutnya terdapat deklarasi queue berupa array q berkapasitas 10001 elemen, bertambah 1 dari maksimal untuk jaga-jaga.

Terdapat fungsi `push` untuk memasukkan State baru ke posisi `tail` lalu menambah nilai `tail`, sebagai contoh pada gambar visualisasi di bawah:



Selanjutnya ada fungsi `pop` untuk mengambil State di posisi `head` lalu menambah nilai `head`, dan fungsi `isEmpty` untuk mengecek apakah queue kosong atau kondisi `head == tail`. Operasi pada queue manual ini memiliki time complexity $O(1)$.

Selanjutnya ada array `grid` untuk menyimpan bentuk labirin, array boolean `visited` sebagai penanda suatu petak sudah pernah dikunjungi dan array `dr` dan `dc` yang berisi perubahan koordinat atas, bawah, kiri, kanan.

Masuk ke fungsi `main()` dimulai dengan menginput `R`, `C`. Lalu, melakukan nested loop untuk membaca matriks labirin ke dalam array `grid` dan sekaligus menginisialisasi array `visited` dengan nilai `false`. Setelah itu, program menerima input koordinat Start dan Finish dimana `SR`, `SC` adalah Start Row dan Start Column posisi awal, `FR`, `FC` untuk Finish Row, Finish Column untuk posisi tujuan.

Proses BFS dimulai dengan memanggil `push` untuk memasukkan posisi Start ke antrean dengan nilai jarak 0, dan langsung menandai titik Start tersebut pada array `visited` menjadi `true`.

Program menginisialisasi variabel `ans` dengan nilai `-1` jika jalan sudah buntu. Selama `!isEmpty()`, program akan melakukan `pop` State di depan antrean sebagai titik `curr`. Jika titik ini sama dengan titik Finish, program mencatat jaraknya di `ans` dan langsung melakukan `break` untuk menghentikan pencarian, karena BFS jika sudah mencapai target adalah maka itu rute terpendek.

Untuk algoritma ini, time complexity adalah $O(R \times C)$ karena worst case dimana misalnya tidak ada dinding pada labirin, algoritma BFS akan mengunjungi setiap petak di dalam grid tepat satu kali, dan memproses pengecekan 4 arah dengan $O(1)$ per petaknya dan untuk Space complexity-nya adalah $O(R \times C)$ karena algoritma ini memerlukan alokasi memori sebanyak jumlah keseluruhan petak di dalam labirin.

Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 2.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

```
T
```

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$

- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada $(0,0)$ dan posisi tujuan berada pada $(4,5)$. Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel $(0,5)$.

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Tabel 4. Source code prob4.cpp Modul 6

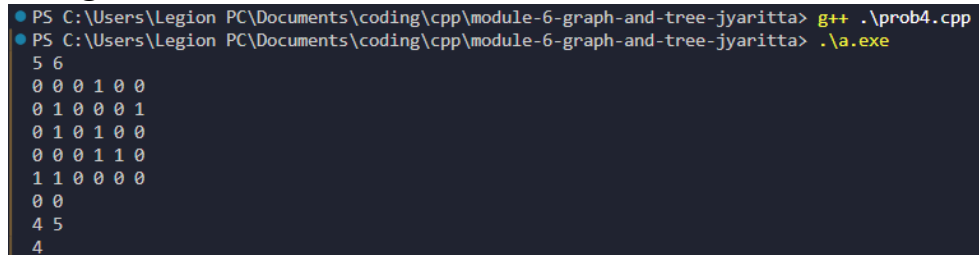
```
1  #include <iostream>
2
3  using namespace std;
4
5  struct Labirin {
6      int grid[10][10];
7      bool visited[10][10];
8      int R, C;
9      int FR, FC;
10     int total_jalur;
11 };
12
13 void dfs(Labirin& map, int r, int c) {
14     if (r == map.FR && c == map.FC) {
15         map.total_jalur++;
16         return;
17     }
18
19     int dr[] = {-1, 1, 0, 0};
20     int dc[] = {0, 0, -1, 1};
21
22     for (int i = 0; i < 4; i++) {
23         int nr = r + dr[i];
24         int nc = c + dc[i];
25
26         if (nr >= 0 && nr < map.R && nc >= 0 && nc <
map.C) {
27             if (map.grid[nr][nc] == 0 &&
!map.visited[nr][nc]) {
28                 map.visited[nr][nc] = true;
29                 dfs(map, nr, nc);
30                 map.visited[nr][nc] = false;
31             }
32         }
33     }
34 }
35
36 int main() {
37     Labirin map;
38     map.total_jalur = 0;
39     cin >> map.R >> map.C;
40
41     for (int i = 0; i < map.R; i++) {
42         for (int j = 0; j < map.C; j++) {
43             cin >> map.grid[i][j];
44             map.visited[i][j] = false;
45         }
46     }
47 }
```

```

48     int SR, SC;
49     cin >> SR >> SC;
50     cin >> map.FR >> map.FC;
51
52     map.visited[SR][SC] = true;
53     dfs(map, SR, SC);
54     cout << map.total_jalur << "\n";
55     return 0;
56 }

```

B. Output Program



```

PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> g++ .\prob4.cpp
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> .\a.exe
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

```

Gambar 4. Screenshot Hasil Jawaban Soal 4 Modul 6

C. Pembahasan

Algoritma berikut menggunakan metode Depth-First Search dengan teknik Backtracking. Cara kerja algoritma ini ia akan memilih satu jalur dan menelusurinya hingga sampai ke tujuan atau setidaknya jalan buntu, lalu mundur untuk mencoba jalur yang lain.

Pertama membuat struct Labirin untuk menaruh data labirinnya di sana. Selanjutnya ada fungsi dfs yang menerima parameter dari struct Labirin beserta posisi baris *r* dan kolom *c* saat ini. Fungsi DFS ini bertugas mengecek apakah sudah sampai di tempat tujuan, melihat ke 4 arah untuk memastikan jalan berikutnya bukan tembok atau tidak keluar batas labirin, sebelum mulai melangkah.

Di dalam fungsi `main()`, pertama program akan menyiapkan peta Labirin, mendeklarasikan total jalur yang ada lalu membaca ukuran baris dan kolom untuk Labirin. Selanjutnya ada perulangan untuk membaca input angka 0 dan 1 ke dalam grid. Lalu program akan melakukan perulangan untuk membaca input angka dan memasukkannya ke dalam grid. Selanjutnya program akan meminta input dimana tempat start dan finish. Sebelum jalan, melakukan tempat awal menjadi `visited = true` agar tidak terjadi kesalahan kembali lagi ke garis start. Setelah itu, panggil fungsi dfs untuk mencari total jalur.

Time complexity algoritma ini adalah $O(4^{R \times C})$ karena dalam kondisi worst case dimana misalnya labirin tanpa dinding sel dapat bercabang hingga ke 4 arah yang berbeda yang menghasilkan banyak jalur yang bertambah secara eksponensial dan space complexity-nya adalah $O(R \times C)$ karena memori hanya digunakan untuk menyimpan array 2D grid dan `visited` di dalam struct dan lainnya.

Soal 5: Tree Traversal

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **Red-Black Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTREE terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- **PREORDER**, yaitu urutan root, subtree kiri, lalu subtree kanan.
- **INORDER**, yaitu urutan subtree kiri, root, lalu subtree kanan.
- **POSTORDER**, yaitu urutan subtree kiri, subtree kanan, lalu root.
- **ALL**, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

```
N
A1 A2 ... AN
Q
T1
T2
...
TQ
```

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTREE.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

```
[Preorder] : daftar_angka
[Inorder]  : daftar_angka
[Postorder]: daftar_angka
```

Jika query adalah **ALL**, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDE	[Preorder] : 50 30 20 40 70 60 80
R	[Inorder] : 20 30 40 50 60 70 80
INORDER	[Postorder] : 20 40 30 60 80 70 50
POSTORD	
ER ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Tabel 5. Source code prob5.cpp Modul 6

```
1  #include <iostream>
2  #include "RedBlackTree.h"
3
4  using namespace std;
5
6  bool isMatch(const char* a, const char* b) {
7      int i = 0;
8      while (a[i] != '\0' && b[i] != '\0') {
9          if (a[i] != b[i]) return false;
10         i++;
11     }
12     return a[i] == '\0' && b[i] == '\0';
13 }
14
15 void Preorder(const RedBlackTree::Node* node, const
RedBlackTree::Node* nilNode, bool& isFirst) {
16     if (node == nilNode || node->isNil) return;
17
18     if (!isFirst) cout << " ";
19     cout << node->key;
20     isFirst = false;
21
22     Preorder(node->left, nilNode, isFirst);
23     Preorder(node->right, nilNode, isFirst);
24 }
25
26 void Inorder(const RedBlackTree::Node* node, const
RedBlackTree::Node* nilNode, bool& isFirst) {
27     if (node == nilNode || node->isNil) return;
28
29     Inorder(node->left, nilNode, isFirst);
30
31     if (!isFirst) cout << " ";
32     cout << node->key;
33     isFirst = false;
34
35     Inorder(node->right, nilNode, isFirst);
36 }
37
38 void Postorder(const RedBlackTree::Node* node, const
RedBlackTree::Node* nilNode, bool& isFirst) {
39     if (node == nilNode || node->isNil) return;
40
41     Postorder(node->left, nilNode, isFirst);
42     Postorder(node->right, nilNode, isFirst);
```

```

43
44     if (!isFirst) cout << " ";
45     cout << node->key;
46     isFirst = false;
47 }
48
49 int main() {
50     int n;
51     cin >> n;
52
53     RedBlackTree rbt;
54
55     for (int i = 0; i < n; i++) {
56         int val;
57         cin >> val;
58         if (!rbt.contains(val)) {
59             rbt.insert(val);
60         }
61     }
62
63     int q;
64     cin >> q;
65
66     if (rbt.empty()) {
67         cout << "Tree kosong\n";
68         return 0;
69     }
70
71     for (int i = 0; i < q; i++) {
72         char query[20];
73         cin >> query;
74
75         if (isMatch(query, "PREORDER") || isMatch(query,
76 "ALL")) {
77             cout << "[Preorder] : ";
78             bool isFirst = true;
79             Preorder(rbt.root(), rbt.nil(), isFirst);
80             cout << "\n";
81         }
82         if (isMatch(query, "INORDER") || isMatch(query,
83 "ALL")) {
84             cout << "[Inorder] : ";
85             bool isFirst = true;
86             Inorder(rbt.root(), rbt.nil(), isFirst);
87             cout << "\n";
88         }
89     }
90 }

```

```

88         if (isMatch(query, "POSTORDER") || isMatch(query,
89 "ALL")) {
90             cout << "[Postorder]: ";
91             bool isFirst = true;
92             Postorder(rbt.root(), rbt.nil(), isFirst);
93             cout << "\n";
94         }
95     }
96     return 0;
97 }

```

B. Output Program

```

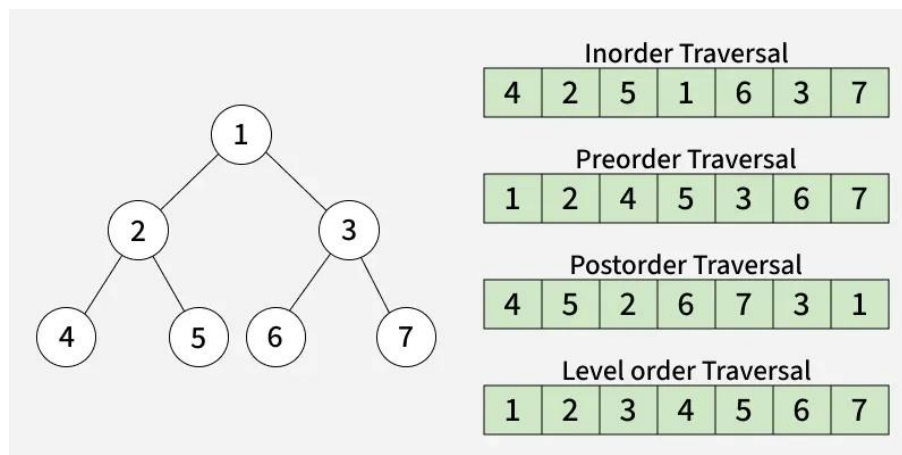
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> g++ .\prob5.cpp .\RedBlackTree.cpp -o prob5
PS C:\Users\Legion PC\Documents\coding\cpp\module-6-graph-and-tree-jyaritta> .\prob5.exe
7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder]: 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder]: 20 40 30 60 80 70 50

```

Gambar 5. Screenshot Hasil Jawaban Soal 5 Modul 6

C. Pembahasan

Kode prob5.cpp merupakan implementasi algoritma struktur data Red-Black Tree berupa Preorder, Inorder, dan Postorder berdasarkan query yang diminta. Singkatnya, penjelasan sederhananya ada di gambar berikut:



Fungsi `isMatch`, digunakan untuk membandingkan dua string tanpa menggunakan library `<string>` dengan melakukan iterasi `while` untuk mengecek karakter satu per satu. Jika ada yang berbeda, kembalikan `false`, ini dipakai untuk mengecek query user seperti "PREORDER", "INORDER", "POSTORDER".

Fungsi `Preorder`, menelusuri tree dari root ke kiri lalu ke kanan dengan membuat base case jika node sekarang adalah `nilNode` atau `isNil` yakni kosong, maka return. Kalau bukan, cetak nilai `node->key`. Terdapat juga pengecekan boolean `isFirst` untuk memastikan spasi saat pencetakan nanti. Lalu rekursif ke kiri, dan rekursif lagi ke kanan.

Fungsi `Inorder`, menelusuri tree dari kiri ke root lalu ke kanan, cara kerjanya mirip dengan `preorder`, tetapi urutan eksekusinya adalah rekursif ke kiri dahulu, lalu cetak nilai `node->key`, baru rekursif ke kanan.

Fungsi `Postorder`, menelusuri tree dari kiri ke kanan lalu ke root dengan rekursif ke kiri terlebih dahulu, lalu rekursif ke kanan, dan diakhiri dengan mencetak nilai `node->key`.

Ketiga fungsi traversal ini memiliki time complexity $O(N)$ karena setiap node di dalam tree dikunjungi satu kali dan space complexity $O(\log N)$ yang berasal dari call stack memori untuk fungsi rekursi.

Selanjutnya pada fungsi `main()`, pertama-tama melakukan inout untuk `n` dan mendeklarasikan `RedBlackTree` sebagai `rbt`. Selanjutnya program akan melakukan perulangan `n` kali untuk membaca angka dan setiap angka dicek menggunakan `rbt.contains(val)` agar bilangan duplikat diabaikan. Lalu nilai tersebut di-insert ke dalam tree. Selanjutnya program menerima input `q` untuk jumlah query. Jika tree kosong program mencetak pesan "Tree kosong" dan return 0 untuk berhenti. Jika tidak kosong, program melakukan iterasi sebanyak `q` kali untuk membaca string query dan menggunakan perulangan menggunakan fungsi `isMatch`, dengan mencocokkan input dengan "PREORDER", "INORDER", "POSTORDER", atau "ALL". Jika cocok, setiap kondisi akan memanggil fungsi traversal yang tepat dengan `isMatch`.

Time complexity keseluruhan program ini adalah $O(N \log N + Q \times N)$ karena memasukkan `N` elemen ke dalam RBT memakan sebesar $O(\log N)$ per elemennya, dan setiap eksekusi dari `Q` query traversal $O(N)$. lalu, space complexity nya adalah $O(N)$ karena memerlukan alokasi memori untuk menyimpan `N` elemen di dalam node `RedBlackTree`.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-jyaritta>